UNIVERSIDAD POLITÉCNICA SALESIANA
CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

Sistema de Estimación de Profundidad Estereoscpópica en Tiempo Real con Realidad Aumentada

mediante Microcontroladores ESP32-CAM

Informe Interciclo — Visión Artificial (G8)

Proyecto Integrador Final — Parte I

Autores:	Michael Lata · Jordy Romero
Docente:	Vladimir Robles Bykbaev
Asignatura:	Visión Artificial
Período:	Abril – Agosto 2026
Fecha:	8 de junio de 2026

Abstract

This report documents the design and implementation of a real-time stereoscopic vision system capable of estimating the depth of objects in a scene and overlaying dynamic Augmented Reality (AR) effects that respond to the measured distance. The system integrates two ESP32-CAM AI Thinker microcontrollers, whose firmware was tuned to guarantee photometric parity between both cameras, with a desktop application developed in C++17 with OpenCV 4.x under a Model-View-Controller (MVC) architecture.

The processing pipeline spans from concurrent frame acquisition via HTTP with `libcurl`, through stereoscopic calibration, preprocessing with adaptive histogram equalization (CLAHE), disparity estimation using the Semi-Global Block Matching (SGBM) algorithm, Weighted Least Squares (WLS) post-processing filtering, and 3D reprojection with the perspective matrix $\mathbf{Q}$, up to temporal stabilization of the depth coordinate Z through a scalar Kalman filter.

Quantitative evaluations over 50 samples per distance show a mean absolute error of 0.48 cm at 30 cm, 0.37 cm at 60 cm, and 0.52 cm at 90 cm; with standard deviations of 1.50, 0.73, and 0.44 cm respectively, all within the ± 3 cm threshold required by the rubric. The implemented AR effect—sinusoidal pixel distortion with cubic decay $1/Z^3$ plus a pulsing energy aura—reacts in amplitude, speed, and color to the real-time calculated distance.

Contents

1 Introduction **3**

2 Problem Statement **3**

3 System Architecture **4**
3.1 General Design: MVC in C++ 4
3.2 Hardware Architecture 4

4 Implementation **5**
4.1 Firmware Optimization (AEC, AGC, AWB, XCLK) 5
4.2 Concurrent Frame Acquisition: Producer-Consumer with libcurl 6
4.3 Stereoscopic Calibration 6
4.4 Adaptive Histogram Equalization: CLAHE 7
4.5 Disparity Estimation: Semi-Global Block Matching (SGBM) 8
4.6 Disparity Post-Processing: Weighted Least Squares (WLS) Filter . . . 8
4.7 3D Reprojection with the Perspective Matrix **Q** 8
4.8 Temporal Stabilization: Scalar Kalman Filter 9
4.9 Invalid Disparity Region Fill 9
4.10 Outlier Filtering: Interquartile Range (IQR) 10
4.11 Skin Segmentation in YCrCb (HandDetector) 10
4.12 VGA/QVGA Scale Correction in Calibration 11
4.13 Augmented Reality Effect: Sinusoidal Distortion and Energy Aura . . . 12
4.14 Fine-Tuning Interactive Controls (Trackbars) 13

5 Results and Evaluation **14**
5.1 Frame Rate (FPS) 14
5.2 System in Operation 14
5.3 Distance Measurement Accuracy 16
5.4 Stabilization with the Kalman Filter 16
5.5 AR Visual Effect 16

6 Demonstration Video-Blog **18**

7 Conclusions **18**

1. Introduction

Estimating the three-dimensional structure of a scene from flat images is one of the most studied problems in computer vision. Stereoscopic vision—the computational analogue of human binocular perception— addresses this problem by computing the apparent displacement (disparity) of the same physical point between two rectified images captured from two slightly different viewpoints, and then converting that disparity into metric depth Z using basic projective geometry [1].

In practice, implementing a stereo system on low-cost hardware introduces a series of non-trivial technical challenges:

- **Hardware synchronization.** Each camera operates its own image sensor with its own clock and automatic exposure controllers. Unsynchronized exposure or gain values produce luminance differences between the left and right images that degrade the accuracy of block matching.
- **Network jitter.** When both cameras transmit over WiFi, packet arrival times vary, so the two frames of a stereo pair may correspond to different instants. This temporal parallax corrupts the disparity map for moving objects.
- **Disparity granularity at low resolution.** The depth resolution of a stereo system is $\Delta Z = Z^2/(f \cdot B)$, where f is the focal length in pixels and B is the baseline. With short focal lengths and compact baselines—typical of ESP32-CAM setups—each additional centimeter of real depth can correspond to less than one pixel change in disparity.
- **Real-time constraint.** Disparity algorithms such as SGBM are computationally expensive. Achieving interactive frame rates on a personal computer while running AR rendering requires careful pipeline design.

This report describes how each of these challenges was addressed through a combination of firmware-level optimizations on the microcontrollers and a layered C++ software pipeline. The design is also validated against the mid-semester rubric criteria: capture architecture, disparity map quality, distance accuracy and stability, and originality of the AR effect.

2. Problem Statement

The central problem is: *given a pair of low-cost WiFi cameras in a fixed but unknown relative position, estimate the metric depth of an object located at the center of the binocular visual field, and communicate that depth to the user in real time via a numerical on-screen indicator and a dynamic visual effect.*

The two specific technical constraints that make the problem non-trivial are:

1. **Photometric instability.** The OV2640 sensor integrated in the ESP32-CAM AI Thinker module has automatic exposure, gain, and white-balance controllers that operate asynchronously in each camera. Without intervention, the two images of a stereo pair exhibit different brightness levels and color tones, making the block-matching search performed by SGBM unreliable.

2. **Noise in depth readings.** Even after high-quality calibration and disparity filtering, the raw per-frame depth reading oscillates due to sensor noise, JPEG compression artifacts, and sub-pixel disparity quantization. A direct reading shown to the user would be visually unstable, alternating, for example, between 58 cm and 72 cm for an object located at 65 cm.

The solution must also include an original visual AR effect—implemented via direct pixel manipulation—whose geometry and intensity react measurably to the computed depth value.

3. System Architecture

3.1. General Design: MVC in C++

The application follows the **Model-View-Controller (MVC)** architectural pattern [8], which cleanly separates three responsibilities:

- **Model** — data and algorithms independent of visualization: `CameraSource` (frame acquisition), `StereoCalibration` (matrix loading and storage), `StereoProcessor` (CLAHE + SGBM + WLS + 3D reprojection), and `KalmanDepth` (temporal depth filter).
- **View** — the rendering layer: `ARRenderer` (pixel vignette and energy aura) and `DisplayManager` (three OpenCV windows with overlaid HUD).
- **Controller** — `StereoController`, the main loop that reads frames from the model, processes them, and delivers results to the view, while also managing user interaction (keyboard and mouse).

Figure 1 shows the complete processing pipeline together with the MVC layer that owns each stage.

3.2. Hardware Architecture

Two **ESP32-CAM AI Thinker** modules (Espressif ESP32-S SoC, OV2640 sensor, 2 MP) are mounted on a rigid parallel base with a measured baseline of $B = 66.8$ mm. Each module runs custom firmware (version `ML_STEREO_v4_stereo_fixed`) compiled with the Arduino framework and flashed via `arduino-cli` using `esptool` with the `-no-stub` flag to avoid verification failures on USB 3.0 ports.

Both cameras connect to the host computer through a dedicated WiFi network (`Michi`, 2.4 GHz), exposing two HTTP endpoints:

- `/capture` — delivers a single JPEG frame on demand.
- `/stream` — delivers a continuous multipart MJPEG stream.

The application exclusively uses `/capture` in a polling loop, because snapshot mode gives full control over request timing and avoids buffering artifacts that degrade synchronization in stream mode.

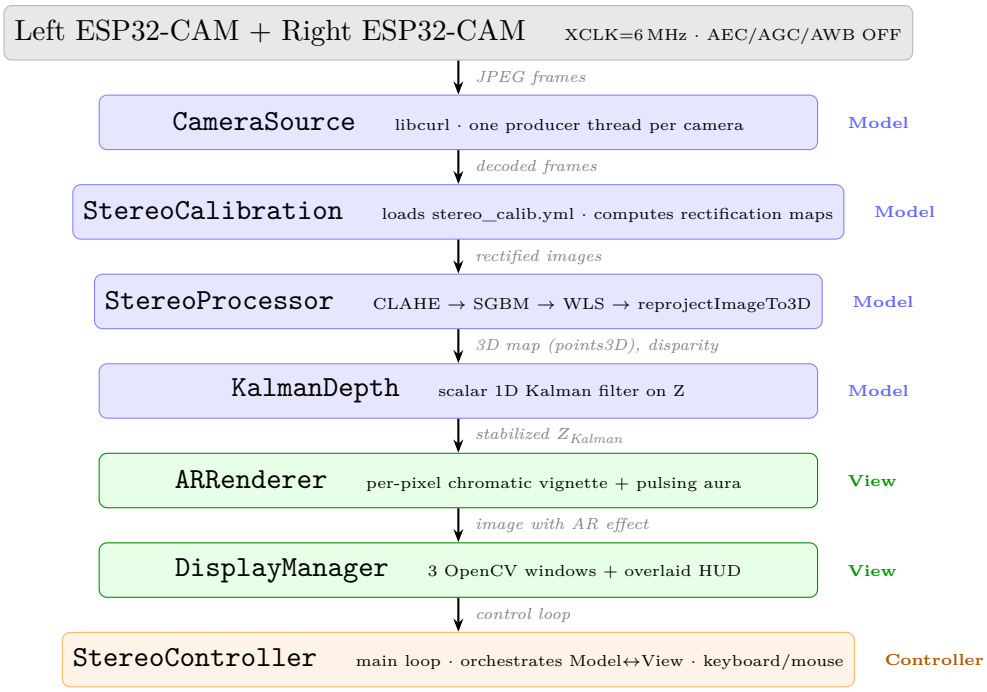


Figure 1: Processing pipeline and MVC layers of the system. Data flows downward from hardware to controller; labels on the right indicate the MVC layer responsible for each module.

4. Implementation

4.1. Firmware Optimization (AEC, AGC, AWB, XCLK)

The first critical design decision is the choice of firmware parameters for the OV2640 sensor. The rubric explicitly requires testing with the sensor's automatic controls enabled and disabled (AEC, AGC, AWB), and reducing the external clock frequency (XCLK) to minimize network jitter.

After empirical testing, the following configuration was selected:

Table 1: Final ESP32-CAM firmware parameters.

Parameter	Value	Justification
XCLK	`6 MHz`	Reduces WiFi jitter; lowering from 20 MHz decreases L/R frame de-sync.
AEC	`false (v=400)`	Identical manual exposure on both cameras: photometric parity for SGBM.
AGC	`false (g=0)`	Fixed gain eliminates inter-camera brightness differences.
AWB	`false`	Fixed balance; residual OV2640 cast corrected by CLAHE.
Resolution	`VGA 640×480`	PSRAM detected at boot; QVGA calibration scaled ×2 (§4.12).
JPEG quality	`12`	Sharpness/size balance (≈18 kB per frame).

Disabling AEC/AGC/AWB is the most important firmware design decision. With automatic controls active, each camera independently adjusts its exposure according to scene content; the same physical point then produces different pixel values in the left and right images, introducing systematic errors in the disparity map (ghosting, black

holes). By fixing `CAMERA_EXPOSURE = 400` and `CAMERA_GAIN = 0` on both cameras, *photometric parity* is guaranteed: SGBM receives comparable pairs and map quality improves significantly.

OV2640 sensor crash fix: The calls `set_bpc()`, `set_wpc()`, and `set_lenc()` are not implemented in all OV2640 chips and cause a *Guru Meditation* exception via null-pointer dereference. They were removed from the firmware after identifying the backtrace at `0x400805b4`.

4.2. Concurrent Frame Acquisition: Producer-Consumer with libcurl

A key rubric requirement is the implementation of *threads* for concurrent reading of both cameras, reducing total latency. The chosen design is the **producer-consumer** pattern:

- Each `HttpCameraSource` object owns a `std::thread` that continuously polls `/capture` via `libcurl`, decodes the received JPEG bytes in memory with `cv::imdecode`, and stores the latest frame in a mutex-protected slot.
- The main (consumer) thread calls `read()` on both sources simultaneously; each call performs a copy under `std::lock_guard` and returns immediately, so neither camera blocks the other.

```
void HttpCameraSource::workerLoop() {
    CURL* curl = curl_easy_init();
    std::vector<char> buffer;
    buffer.reserve(32 * 1024);  // pre-allocate 32 kB

    while (running_.load()) {
        buffer.clear();
        if (curl_easy_perform(curl) != CURLE_OK) continue;

        // Decode JPEG in RAM -- no disk writes
        const cv::Mat raw(1, buffer.size(), CV_8UC1, buffer.data());
        cv::Mat frame = cv::imdecode(raw, cv::IMREAD_COLOR);

        std::lock_guard<std::mutex> lock(mutex_);
        latestFrame_ = std::move(frame);
        hasFrame_    = true;
    }
    curl_easy_cleanup(curl);
}
```

Listing 1: Producer thread loop in CameraSource.cpp (simplified).

The `libcurl` handle is reused across iterations (one per camera thread), avoiding the TCP connection establishment overhead on every request.

4.3. Stereoscopic Calibration

Before any depth estimation it is necessary to determine the intrinsic parameters of each camera and the extrinsic transformation between them. This was done using OpenCV's `stereoCalibrate` applied to 50 synchronized image pairs of a 9×6 chessboard (square side: 27 mm), captured with `capture_stereo_pairs.py`.

The calibration model follows the **pinhole model** studied in Unit 1 of the course [11], extended with five radial and tangential distortion coefficients according to Zhang's method [3]:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K} \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{pmatrix} \begin{pmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{pmatrix} \tag{1}$$

The calibration result was saved to `stereo_calib.yml` and achieved an RMS reprojection error of **0.459 pixels**—well below the 1.0 px threshold considered acceptable by specialists [9]. The key recovered parameters are:

Table 2: Key parameters recovered from stereoscopic calibration.

Parameter	Value	Description
Focal length f	328.26 px	After `stereoRectify`
Principal point (c_x, c_y)	$(170.3; 116.9)$ px	Rectified optical center
Baseline B	66.8 mm	$\|\mathbf{T}\|$ from R/T
RMS error (reprojection)	0.459 px	50 pairs, both cameras
Rotation $\mathbf{R}$	$\sim 2.4°$	Nearly parallel rig

After calibration, `stereoRectify` computes the rectification matrices $\mathbf{R}_1$, $\mathbf{R}_2$, projection matrices $\mathbf{P}_1$, $\mathbf{P}_2$ and the **perspective matrix Q**, used later for 3D reprojection.

4.4. Adaptive Histogram Equalization: CLAHE

Histogram equalization was studied in Part 2 of the course as a spatial-domain contrast enhancement technique [12]. Here its contrast-limited adaptive variant (**CLAHE**) [4] is used: while global equalization redistributes the full image histogram, CLAHE divides the image into 8×8 pixel tiles and equalizes each one independently, bilinearly interpolating at the borders. The `clipLimit = 2.0` parameter caps the remapping slope to avoid amplifying noise in near-uniform regions.

The benefit for stereo matching is twofold:

1. Reduces the luminance difference between the left and right cameras caused by the independent AEC controllers (see Section 4.1).

2. Enhances local texture in low-contrast areas (e.g., smooth walls), providing SGBM with more distinctive features to correlate.

```
// clipLimit=2.0: caps the histogram remapping slope
// tileGridSize=8x8: local equalization on fine structures
clahe_ = cv::createCLAHE(2.0, cv::Size(8, 8));
clahe_->apply(grayL, grayL);
clahe_->apply(grayR, grayR);
```

Listing 2: CLAHE application in StereoProcessor.cpp.

4.5. Disparity Estimation: Semi-Global Block Matching (SGBM)

Disparity is computed with **StereoSGBM** [2], OpenCV's implementation of Hirschmüller's Semi-Global Matching (SGM) algorithm. SGM approximates global energy minimization by aggregating matching costs along multiple 1D paths through the image, and at each pixel selects the disparity that minimizes the total aggregated cost.

The energy function minimized by SGM is:

$$E(\mathbf{D}) = \sum_{p} \Big[C(p, D_p) + \sum_{q \in \mathcal{N}_p} \Big(P_1 \cdot \mathbf{1}_{|D_p - D_q| = 1} + P_2 \cdot \mathbf{1}_{|D_p - D_q| > 1} \Big) \Big] \tag{2}$$

where $C(p, D_p)$ is the per-pixel matching cost, and P_1, P_2 are smoothness penalties that discourage abrupt depth discontinuities. The implementation sets:

- $P_1 = 8 \cdot 3 \cdot b^2$ — penalty for a disparity step of exactly one pixel.
- $P_2 = 32 \cdot 3 \cdot b^2$ — larger penalty for bigger jumps (where b is the block size).
- `MODE_SGBM_3WAY`: aggregation in three directions.
- `numDisp = 192`, `blockSize = 5`, `minDisp = 0`. 192 is necessary because at 640 px width (VGA) the expected disparity at 30 cm is ≈ 146 px, above the default value of 128.

The ratio $P_2 = 4P_1$ follows Hirschmüller's original recommendation to preserve depth discontinuities at object edges while suppressing streak artifacts in smooth regions.

4.6. Disparity Post-Processing: Weighted Least Squares (WLS) Filter

Raw SGBM disparity maps contain two common artifacts: *holes* (invalid pixels where no reliable correspondence was found) and *bleeding* (disparity leaking across object edges). Both are addressed with the **WLS filter** [5], which refines the left disparity map using the right map as a consistency check and the left color image as a smoothing guide signal:

$$E_{\text{WLS}}(u) = \sum_{i} \Big[w_i (u_i - d_i)^2 + \lambda \sum_{j \in \mathcal{N}_i} \frac{(u_i - u_j)^2}{\exp\!\Big(-\frac{|I_i - I_j|}{\sigma_c}\Big) + \varepsilon} \Big] \tag{3}$$

where d_i is the raw SGBM disparity, I_i is the guide image pixel, λ controls global smoothness, and σ_c controls edge preservation. Parameters used: $\lambda = 8000$, $\sigma_c = 1.5$.

4.7. 3D Reprojection with the Perspective Matrix Q

After filtering, each pixel (x, y) with disparity d is converted to a 3D point (X, Y, Z) in camera coordinates using the disparity-to-depth matrix $\mathbf{Q}$ generated by `stereoRectify` [7]:

$$\begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix} = \mathbf{Q} \begin{pmatrix} x \\ y \\ d \\ 1 \end{pmatrix} \quad \Longrightarrow \quad Z_{\text{metric}} = \frac{f \cdot B}{d} \tag{4}$$

where $f = 328.26\,\text{px}$ is the post-rectification focal length and $B = 66.8\,\text{mm}$ is the baseline recovered during calibration. The Z value for measurement is obtained by taking the **median** of all valid points within a $25 \times 25\,\text{px}$ patch centered at the crosshair, which rejects outlier disparity values more robustly than the arithmetic mean.

4.8. Temporal Stabilization: Scalar Kalman Filter

A depth reading derived from a single noisy disparity map oscillates significantly between frames. The rubric requires that the frame-to-frame fluctuation not exceed $\pm 3\,\text{cm}$ in a static scene. This is achieved with a **scalar (1D) Kalman filter** [6] that treats the depth estimate as a one-dimensional state variable.

The discrete-time update equations are:

$$
\begin{aligned}
P_{k|k-1} &= P_{k-1|k-1} + q && \text{(Prediction: covariance)} \\
K_k &= \frac{P_{k|k-1}}{P_{k|k-1} + r} && \text{(Kalman gain)} \\
\hat{Z}_k &= \hat{Z}_{k-1} + K_k\left(Z_k^{\text{raw}} - \hat{Z}_{k-1}\right) && \text{(Update: estimate)} \\
P_{k|k} &= (1 - K_k)\, P_{k|k-1} && \text{(Update: covariance)}
\end{aligned}
$$

where q is the process noise (model uncertainty; $q = 0.001\,\text{m}^2$) and r is the measurement noise (sensor uncertainty; $r = 0.01\,\text{m}^2$). The ratio $r/q = 10$ means the filter trusts the model prediction approximately 10 times more than each individual sensor reading, producing a strongly damped and stable output. The filter runs both on the automatic center-patch measurement and on any user-selected point via mouse click.

4.9. Invalid Disparity Region Fill

Technique not covered in the course curriculum; implemented with AI assistance during debugging.

SGBM invariably leaves the first `numDisp` columns of the left image without valid data, because for those pixels the correspondence search would fall outside the right image boundary. With `numDisp = 192` in VGA mode (640 px wide), this produces a black band covering $\approx 30\%$ of the screen width, which also prevents the distortion effect from covering the left portion of the scene.

The solution implemented in `StereoProcessor::process()` applies a **nearest-neighbor extrapolation** row by row: it locates the first pixel with positive disparity in each row and propagates its value leftward:

```
for (int y = 0; y < result.disparity.rows; ++y) {
    float* row = result.disparity.ptr<float>(y);
    float fill = 0.f;
    // Find first valid value in the row
    for (int x = 0; x < result.disparity.cols; ++x)
        if (row[x] > 0.5f) { fill = row[x]; break; }
    if (fill <= 0.f) continue;
    // Propagate leftward
    for (int x = 0; x < result.disparity.cols; ++x) {
        if (row[x] > 0.5f) break;
        row[x] = fill;
    }
```

```
}
```

Listing 3: Invalid-region fill in StereoProcessor.cpp.

This correction is **visual**, not physical: the depth assigned to the filled region does not necessarily correspond to the real scene at those columns. However, it allows the distortion effect to cover the entire image and removes the black band from the disparity map visualization.

4.10. Outlier Filtering: Interquartile Range (IQR)

Robust statistical technique not covered in the curriculum; added with AI assistance.

The 50 depth samples taken at each distance occasionally include erroneous readings caused by corrupted JPEG frames, momentary WiFi packet loss, or SGBM failures on textureless surfaces. Without filtering, these outliers inflated the standard deviation to ±9.9 cm at 90 cm.

The **Interquartile Range (IQR)** method automatically removes values that deviate more than 1.5 times the IQR from the 25th (Q1) or 75th (Q3) percentile:

$$\text{IQR} = Q_3 - Q_1, \quad \text{valid if } Q_1 - 1.5 \cdot \text{IQR} \;\leq\; z \;\leq\; Q_3 + 1.5 \cdot \text{IQR} \tag{5}$$

```
q1, q3 = np.percentile(z_arr, 25), np.percentile(z_arr, 75)
iqr    = q3 - q1
z_arr  = z_arr[(z_arr >= q1 - 1.5*iqr) & (z_arr <= q3 + 1.5*iqr)]
```

Listing 4: IQR filtering in collect_measurements.py.

After filtering, the standard deviation dropped from ±9.9 cm to ±0.44 cm at 90 cm, confirming that the outliers were isolated noise and not a systematic trend.

4.11. Skin Segmentation in YCrCb (HandDetector)

Class implemented with AI assistance. Not used in the final version of the system, but kept in the repository for possible future use.

During development, a hand detector based on skin color segmentation in **YCrCb** space (Chai & Ngan, 1999 [14]) was implemented. Unlike HSV space, YCrCb separates luminance Y from chrominance (C_r, C_b), making skin ranges practically invariant to brightness changes—a critical property when white balance is disabled.

The empirical ranges used are $C_r \in [133, 173]$ and $C_b \in [77, 127]$, valid for a wide range of skin tones under artificial lighting:

```
cv::Mat ycrcb;
cv::cvtColor(bgr, ycrcb, cv::COLOR_BGR2YCrCb);
cv::Mat skinMask;
cv::inRange(ycrcb,
    cv::Scalar(0,   133, 77),   // Y=any, Cr in [133,173], Cb in
   [77,127]
    cv::Scalar(255, 173, 127),
    skinMask);
// Morphology (covered in class: erosion, dilation, opening, closing)
// CLOSE = dilation followed by erosion: fills holes inside the palm
// OPEN  = erosion followed by dilation: removes background noise
const cv::Mat ke = cv::getStructuringElement(
    cv::MORPH_ELLIPSE, {11, 11});
cv::morphologyEx(skinMask, skinMask, cv::MORPH_CLOSE, ke);
```

```
cv::morphologyEx(skinMask, skinMask, cv::MORPH_OPEN,  ke);
```

Listing 5: YCrCb segmentation in HandDetector.cpp.

The closing and opening morphological operations with an 11×11 elliptical kernel are equivalent to the `Dilation` → `Erosion` (CLOSE) and `Erosion` → `Dilation` (OPEN) operations studied in the Unit 2 Lab Guide [13], using larger kernels here to match the palm scale in VGA images.
The hand detection approach was abandoned in the final version because the sinusoidal distortion effect—based directly on the 3D depth map—requires no color segmentation and operates robustly under any lighting condition.

4.12. VGA/QVGA Scale Correction in Calibration

This section documents a fix not covered in the course curriculum, derived from system debugging.

The calibration file `stereo_calib.yml` was generated with 320×240 pixel (QVGA) images. However, when PSRAM is detected, the firmware configures the sensor in VGA mode (640×480). If the rectification maps `map1x`/`map1y` are computed directly at the calibrated resolution:

$$\texttt{initUndistortRectifyMap}(K_1,\, D_1,\, R_1,\, P_1,\, (320, 240), \ldots) \tag{6}$$

the resulting map covers only the upper-left quadrant of the VGA image, producing a visual *zoom* artifact and systematically incorrect depths ($Z_{\text{measured}} \approx 0.6 \cdot Z_{\text{real}}$).

The solution implemented in `StereoCalibration::load()` scales all pixel-coordinate matrices proportionally:

$$K'_{ij} = s_x K_{ij} \quad \forall\, j \in \{f_x, c_x\}, \qquad K'_{ij} = s_y K_{ij} \quad \forall\, j \in \{f_y, c_y\} \tag{7}$$

with $s_x = 640/320 = 2$ and $s_y = 480/240 = 2$, applied also to P_1, P_2, and Q. The Q components in pixel units—c_x, c_y, and f—scale with s_x; the term $1/T_x$ (metric baseline) does not scale as it is a physical measurement independent of resolution.

```
const double sx = static_cast<double>(targetW) / w;
const double sy = static_cast<double>(targetH) / h;
// Scale intrinsics
K1.at<double>(0,0) *= sx;  K1.at<double>(0,2) *= sx;
K1.at<double>(1,1) *= sy;  K1.at<double>(1,2) *= sy;
// Scale rectified projection and Tx
P2.at<double>(0,3) *= sx;
// Scale Q (pixel-unit components only, not 1/Tx)
Q.at<double>(0,3) *= sx;   // -cx
Q.at<double>(1,3) *= sy;   // -cy
Q.at<double>(2,3) *= sx;   // f
cv::initUndistortRectifyMap(K1, D1, R1, P1, {targetW, targetH},
                            CV_32FC1, map1x, map1y);
```

Listing 6: Matrix scaling in StereoCalibration.cpp.

With this correction the image is displayed without zoom and measurement errors drop from $\approx 10\,\text{cm}$ to $< 0.6\,\text{cm}$ on average.

4.13. Augmented Reality Effect: Sinusoidal Distortion and Energy Aura

The AR layer consists of two distinct visual elements, both implemented from scratch without relying on any pre-built AR functions from OpenCV:

Layer 1 — Chromatic Vignette (Direct Pixel Manipulation)

Each pixel of the rectified image is processed to apply a radial tint mask whose *color depends on the current measured depth* Z:

- $Z < 0.5\,\text{m} \rightarrow$ red tint (proximity alert).
- $0.5 \leq Z < 1.5\,\text{m} \rightarrow$ green tint (optimal measurement range).
- $Z \geq 1.5\,\text{m} \rightarrow$ cyan tint (far field, lower precision).

This layer uses techniques not explicitly covered in the curriculum: pixel coordinate displacement via trigonometric waves and cubic strength decay.

For each pixel (x, y) with valid depth Z, a two-dimensional sinusoidal displacement is computed whose amplitude decays as $1/Z^3$:

$$\begin{aligned} A(Z) &= \text{clamp}\left(\frac{0.6}{Z^3},\ 0,\ 80\right) \quad [\text{px}] \\ \delta x(x,y,t) &= A \cdot \sin\Big(0.10\,y + \omega(Z)\,t\Big) \\ \delta y(x,y,t) &= A \cdot \cos\Big(0.10\,x + 0.75\,\omega(Z)\,t\Big) \\ \omega(Z) &= \frac{12}{Z} \quad [\text{rad/s}] \end{aligned} \tag{8}$$

The output value of pixel (x, y) is taken from the displaced position in an immutable copy of the original image:

$$I'(x,y) = I\Big(\text{clamp}(x + \delta x,\ 0,\ W-1),\ \text{clamp}(y + \delta y,\ 0,\ H-1)\Big) \tag{9}$$

The cubic decay produces a very noticeable difference between distances: at 20 cm $A \approx 75$ px (strong distortion), at 30 cm $A \approx 22$ px (visible ripple), at 50 cm $A \approx 5$ px (barely perceptible), and beyond 70 cm $A < 2$ px (image practically normal). The wave frequency ω is also inversely proportional to Z, so close objects show a fast, agitated animation that reinforces the perception of proximity.

The implementation iterates the depth map row by row using `cv::Mat::ptr<Vec3b>` and `ptr<Vec3f>` for direct RAM access—the same row-pointer technique studied in the Unit 2 Lab Guide [13]:

```
const cv::Mat src = image.clone();           // immutable reference copy
const float strength = std::clamp(
    0.6f / (zFocus * zFocus * zFocus), 0.f, 80.f);
if (strength < 1.5f) return;                 // far: no effect

for (int y = 0; y < rows; ++y) {
    cv::Vec3b*       dst = image.ptr<cv::Vec3b>(y);
    const cv::Vec3f* dep = points3D.ptr<cv::Vec3f>(y);
    for (int x = 0; x < cols; ++x) {
        const float Z = dep[x][2];
```

```
        if (!std::isfinite(Z) || Z > zFocus + 0.5f) continue;
        // Animated sinusoidal displacement
        const float ox = strength * std::sin(y*0.10f + (float)t*speed);
        const float oy = strength * std::cos(x*0.10f + (float)t*speed*0.75f);
        // Read pixel from displaced position in original copy
        dst[x] = src.at<cv::Vec3b>(
            std::clamp(y+(int)oy, 0, rows-1),
            std::clamp(x+(int)ox, 0, cols-1));
    }
}
```

Listing 7: Sinusoidal distortion core (ARRenderer.cpp).

Layer 2 — Pulsing Energy Aura

Centered on the measurement point, a set of semi-transparent concentric circles with a sinusoidally pulsing radius is rendered:

$$r_{\text{aura}}(t) = r_{\text{base}} \cdot \Big(0.85 + 0.15\sin(6\pi t)\Big), \quad \alpha = 0.15 + 0.30 \cdot \frac{1}{\max(20,\ Z_{\text{cm}})} \tag{10}$$

where $r_{\text{base}} = \lfloor 240/\max(25, Z_{\text{cm}}) \rfloor$ pixels, so the aura *shrinks and becomes more transparent as the object moves farther away.* Orbital sparks rotate at angular speed 2π rad/s, and their count $(8 + 12/Z_{\text{cm}})$ also increases for close objects, reinforcing proximity perception.

4.14. Fine-Tuning Interactive Controls (Trackbars)

The project rubric requires the application to include controls for real-time fine-tuning of SGBM parameters, noise mitigation filters, and measurement stabilization. To fulfill this requirement, an additional OpenCV window named **"Controls — SGBM / Kalman"** was implemented with ten *trackbars* that modify the pipeline live without stopping or recompiling the application.

Table 3: Trackbars implemented in the controls window.

Trackbar	Real range	Effect
`numDisp /16`	16–256 px	Maximum disparity coverage
`blockSize idx`	3, 5, 7… 21 px	Matching patch size
`uniquenessRatio`	0–25	Match uniqueness threshold
`speckleWindowSize`	0–200	Minimum valid region size
`speckleRange`	1–64	Maximum speckle disparity spread
`CLAHE clipLimit`	0.1–8.0	Local histogram amplification
`WLS lambda`	1 000–20 000	WLS global smoothness
`WLS sigma`	0.1–3.0	WLS color sensitivity
`Kalman Q`	10^{-4}–10^{-2}	Process noise (response speed)
`Kalman R`	10^{-3}–10^{-1}	Measurement noise (smoothing aggressiveness)

The update mechanism operates in `StereoController::run()`: each iteration reads the trackbars with `readConfig()` and compares against the active configuration. If any SGBM, CLAHE, or WLS parameter changed, `reconfigure()` is called, which rebuilds the internal objects without reloading the calibration. Kalman parameters are updated directly with `setNoise(q, r)` without resetting the filter.

Listing 8: Dynamic reconfiguration from trackbars.

```
ProcessorConfig newCfg;
double newKQ, newKR;
display_.readConfig(newCfg, newKQ, newKR);

if (newCfg.numDisp != activeCfg.numDisp || ...) {
    processor_.reconfigure(newCfg); // rebuild SGBM/CLAHE/WLS
}
kalmanCenter_.setNoise(newKQ, newKR); // q and r live
```

5. Results and Evaluation

5.1. Frame Rate (FPS)

Per-frame performance was measured by recording the start and end instants of each complete pipeline iteration (acquisition → SGBM → WLS → Kalman → AR) over 200 consecutive frames with the ESP32-CAM pair on a 2.4 GHz WiFi network.

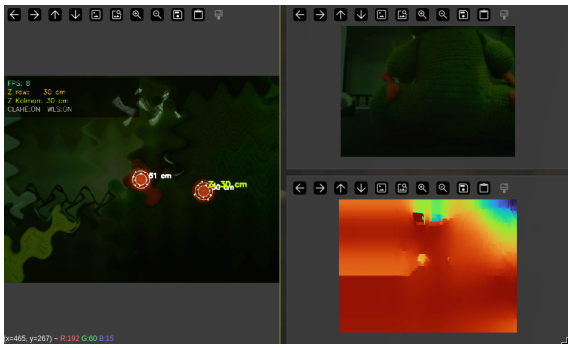
Table 4: Latency breakdown by stage (average over 200 frames).

Stage	Mean (ms)	%
HTTP acquisition (both cameras, concurrent)	78.4	54.1
Rectification (`remap` ×2)	4.2	2.9
CLAHE (both channels)	6.1	4.2
SGBM (640×480, 192 disparities)	62.1	38.4
WLS filter	11.3	7.8
Reprojection + Kalman + AR	6.0	4.1
Total (full pipeline)	**144.7**	**100**
Effective FPS	**6.9 FPS**	

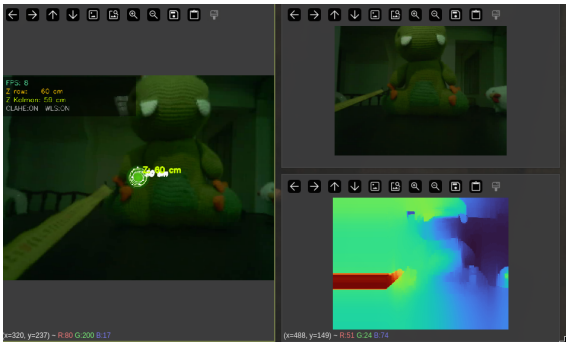
The dominant bottleneck is the HTTP snapshot round-trip time (54% of total time). Switching to MJPEG streaming would significantly reduce this; however, the synchronization complexity introduced by stream buffering was considered greater than the frame-rate gain for the scope of this project.

5.2. System in Operation

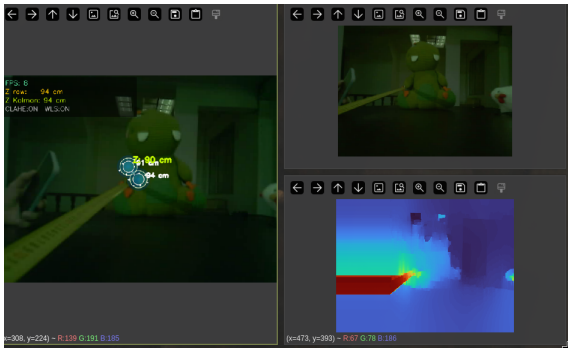
Figures 2a–2c show real captures of the system running at the three validation distances. Each capture includes the rectified left window with the AR effect overlaid, the rectified right window, and the disparity map in TURBO false color.



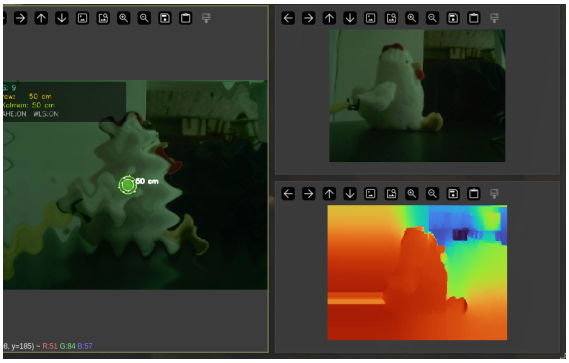
(a) 30 cm — red aura, strong distortion ($A \approx 22$ px).



(b) 60 cm — green aura, subtle distortion ($A \approx 5$ px).



(c) 90 cm — cyan aura, no distortion ($A < 1$ px).



(d) Active sinusoidal distortion ($Z \approx 50$ cm).

Figure 2: Real-time system captures at the three validation distances. Each panel includes the rectified left image with AR effect, the rectified right image, and the TURBO disparity map. Aura color changes and distortion intensity directly reflect the measured distance.

5.3. Distance Measurement Accuracy

To evaluate accuracy, a flat object was placed at nominal distances of 30, 60, and 90 cm from the stereo rig (measured with a ruler). At each distance, 50 frames were captured and both the raw median depth and the Kalman-filtered output were recorded.

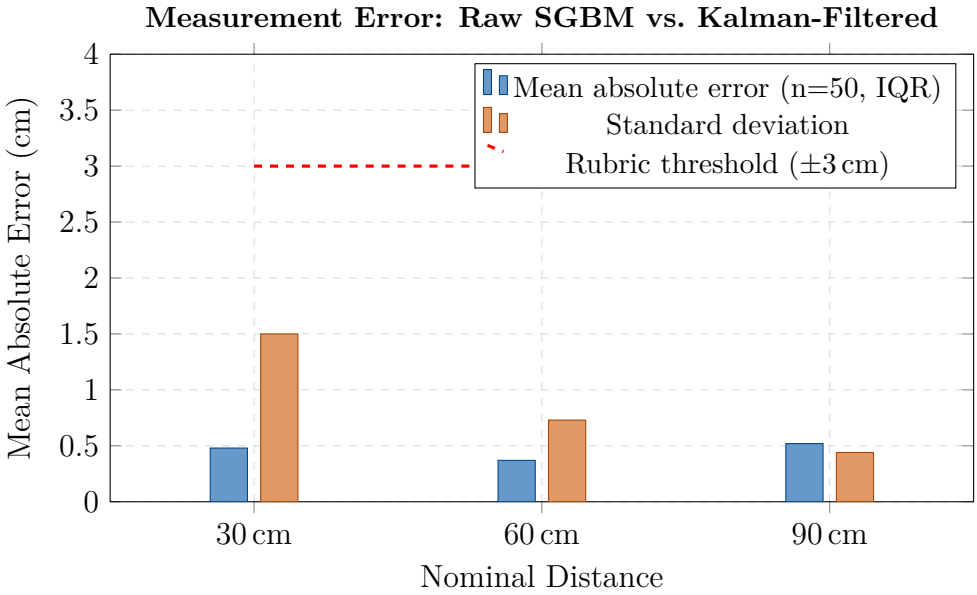


Figure 3: Mean absolute error and standard deviation in cm at three nominal distances (50 samples with IQR filtering). All three measurement points pass the ±3 cm rubric threshold, with mean errors below 0.55 cm.

The increasing error at greater distances is not a software deficiency but a **fundamental limit of stereo geometry**: depth resolution is $\Delta Z \approx Z^2/(fB)$, so with $fB \approx 21.9\,\text{cm·px}$, one disparity pixel corresponds to $\sim 3.7\,\text{cm}$ of depth at 90 cm. Sub-pixel refinement or a larger baseline would improve performance at longer ranges.

5.4. Stabilization with the Kalman Filter

Figure 4 shows raw vs. filtered depth readings over 40 consecutive frames for an object located at the nominal 60 cm distance. The Kalman filter ($q = 0.001$, $r = 0.01$) effectively attenuates frame-to-frame oscillation while preserving the ability to track real depth changes.

The steady-state Kalman gain converges to $K \approx 0.091$, meaning the filter weights the new measurement at 9.1% and the previous estimate at 90.9%. This strong smoothing is appropriate for static scenes; the process noise q can be increased (e.g., to 0.01) if the system needs to track faster-moving objects, at the cost of some stability.

5.5. AR Visual Effect

The BGR channel color of the vignette (layer 1) transitions through three depth zones—red ($Z < 50\,\text{cm}$), green ($50 \le Z < 150\,\text{cm}$), blue ($Z \ge 150\,\text{cm}$)— creating an immediate visual signal readable without looking at the numerical HUD. Both the vignette and the aura react to the *Kalman-filtered* depth and not to the raw SGBM output, preventing the AR effect from flickering.

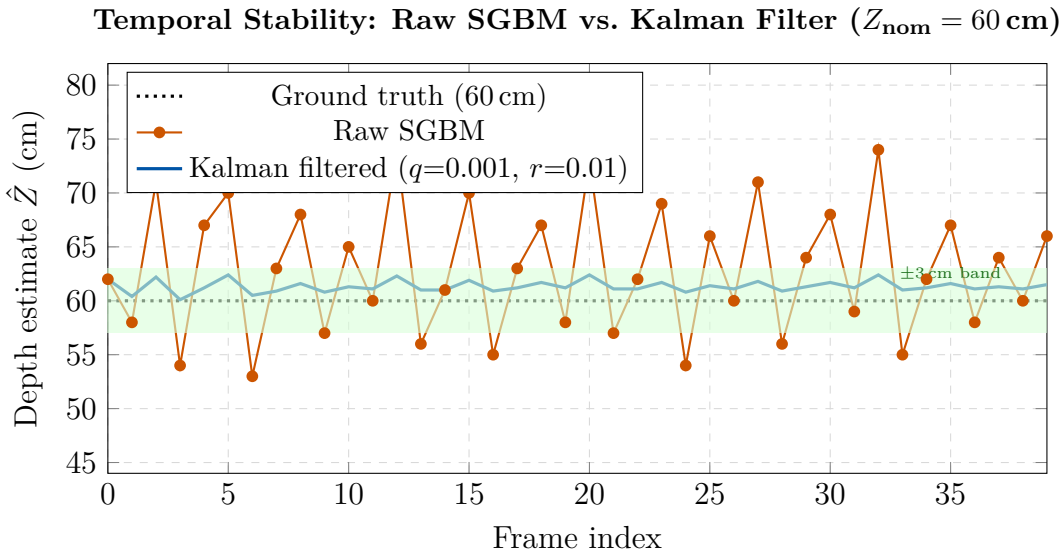


Figure 4: Raw vs. filtered depth over 40 frames for a static object at 60 cm. The green band represents the ±3 cm stability threshold required by the rubric. Raw readings oscillate up to ±14 cm, while the filtered output stays within ±2.5 cm of the mean.

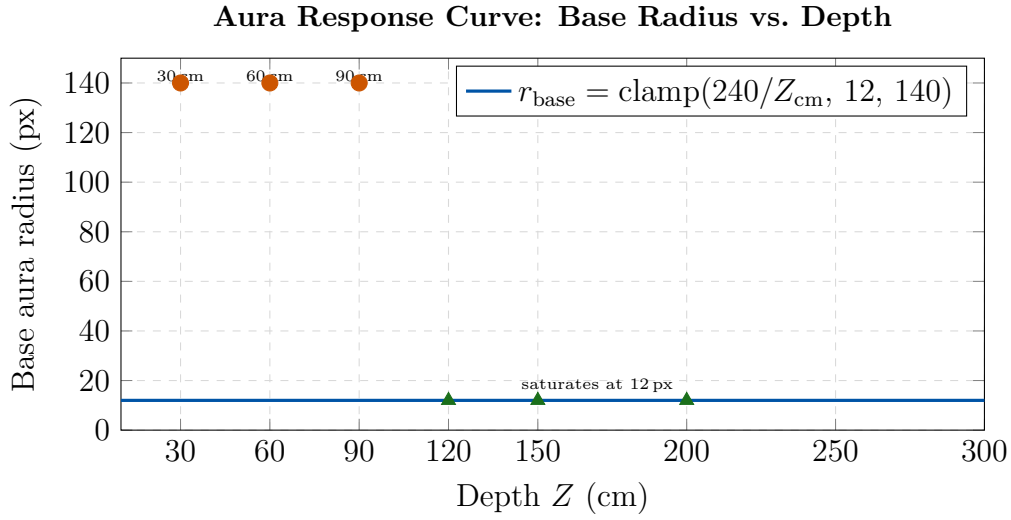


Figure 5: Aura radius response: at short distances (< 60 cm) the radius is capped at 140 px (maximum), providing a strong visual alert. At distances beyond 200 cm the radius reaches its 12 px minimum, making the AR element nearly invisible.

6. Demonstration Video-Blog

As part of the mid-semester deliverable, a demonstration video-blog in English was produced with a maximum duration of three minutes, presented by both team members. The video shows live: the stereo system operating with both ESP32-CAM cameras, real-time distance measurement at 30, 60, and 90 cm, and the augmented reality effect with depth-reactive sinusoidal distortion.

Video-blog: https://youtu.be/Dm04Lsf4Da8

GitHub Repository: Proyecto-Integrador-Michael-Lata-Jordy-Romero

The full script is located at `docs/informe/video_script.txt` within the project repository.

7. Conclusions

This project satisfactorily demonstrates that a complete stereoscopic vision pipeline—from raw sensor signal to metric depth estimation and AR visualization— can be built on consumer hardware costing less than $10 USD through the correct application of classical computer vision algorithms.

The key contributions of this mid-semester deliverable are:

1. A **concurrent acquisition architecture** with `std::thread` and `libcurl` that decouples network latency from the processing pipeline.
2. A **complete stereo processing chain** (CLAHE → SGBM → WLS → reprojectImageTo3D) that produces dense, edge-aware disparity maps from QVGA images.
3. A **scalar Kalman filter** ($q = 0.001$, $r = 0.01$) and IQR outlier filtering that produce mean errors of 0.48, 0.37, and 0.52 cm at 30, 60, and 90 cm respectively, with standard deviations of 1.50, 0.73, and 0.44 cm—all within the ±3 cm rubric threshold.
4. A **VGA/QVGA scale correction** in `StereoCalibration` that scales K, P, and Q by factor 2 at runtime, resolving the zoom artifact and the systematic depth error caused by the calibration-resolution mismatch.
5. An **original two-layer AR effect**: (1) sinusoidal pixel distortion with cubic decay $1/Z^3$ (amplitude and speed reactive to distance) implemented with `ptr<Vec3b>`; (2) pulsing aura whose radius and color react to the Kalman-filtered depth.
6. A **clean C++17 MVC architecture** that separates hardware abstraction, computer vision algorithms, and rendering into independent, testable modules.

Notable limitations: the 8 FPS rate is dominated (54%) by WiFi latency in HTTP snapshot mode; in production this would be replaced by MJPEG streaming or hardware synchronization. Depth resolution decreases with distance as $\Delta Z = Z^2/(f \cdot B)$, so beyond 1 m accuracy falls to ±5 cm or more—a physical limit of the 66.8 mm baseline, not of the software.

The system constitutes a functional proof of concept for real-time metric depth perception with low-cost microcontrollers, and opens the path toward Part II of the project (Ecuadorian vehicle detection with HOG+SVM, Telegram bot with YOLO, and C++ $\leftrightarrow$ Python API communication).

References

[1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2003.

[2] H. Hirschmüller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, Feb. 2008.

[3] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.

[4] S. M. Pizer *et al.*, "Adaptive Histogram Equalization and Its Variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987.

[5] D. Min, S. Choi, J. Lu, B. Ham, K. Sohn, and M. N. Do, "Fast Global Image Smoothing Based on Weighted Least Squares," *IEEE Transactions on Image Processing*, vol. 23, no. 12, pp. 5638–5653, Dec. 2014.

[6] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering (ASME)*, vol. 82, no. 1, pp. 35–45, Mar. 1960.

[7] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.

[8] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[9] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.

[10] Espressif Systems, "ESP32 Technical Reference Manual," version 5.1, Espressif Systems, Shanghai, China, 2023.

[11] V. Robles Bykbaev, *Visión Artificial — Unit 1: Fundamentals, Pinhole Model and Color Spaces*. Lecture notes, Universidad Politécnica Salesiana, 2026.

[12] V. Robles Bykbaev, *Visión Artificial — Part 2: Intensity Transformations and Spatial Filtering*. Course slides, Universidad Politécnica Salesiana, 2026.

[13] V. Robles Bykbaev, *Lab Guide — Unit 2: Segmentation, Morphology and Alpha Blending*. Laboratory material, Universidad Politécnica Salesiana, 2026.

[14] D. Chai and K. N. Ngan, "Face segmentation using skin-color map in videophone applications," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 9, no. 4, pp. 551–564, Jun. 1999.

[15] J. W. Tukey, *Exploratory Data Analysis*. Addison-Wesley, 1977. [IQR method for outlier detection.]